

SWEN90010 - High Integrity Systems Engineering

Programming and Proving with Pointers

Christine Rizkallah



christine.rizkallah@unimelb.edu.au



<http://people.eng.unimelb.edu.au/rizkallahc>

Parameters Passing to Procedures

- Pass by Value
 - Parameters are copied into the procedure, each procedure call operates on its own copy
- Pass by Reference
 - Instead of copying, a reference or pointer to the arguments' memory location is passed, enabling direct access by the procedure to original data

Parameters Passing to Procedures

- Pass by Value
 - Parameters are copied into the procedure, each procedure call operates on its own copy
- Pass by Reference
 - Instead of copying, a reference or pointer to the arguments' memory location is passed, enabling direct access by the procedure to original data

pass by reference can be faster and more memory-efficient compared to pass by value

Pass By Value



```
procedure Pass_By_Value is
  type Rec is record
    X : Integer;
  end record;

  R : Rec;

  procedure Print_R is
  begin
    Put("R.X: "); Put(R.X); New_Line;
  end Print_R;

  procedure CheckR(givenR : in out Rec) is
  begin
    Put("In CheckR"); New_Line;
    givenR.X := R.X + 1;
    Print_R;
  end CheckR;

begin
  R.X := 5;
  CheckR(R);
  Put("CheckR done"); New_Line;
  Print_R;
end Pass_By_Value;
```

Pass By Value

What does this line ->
print?

```
procedure Pass_By_Value is
  type Rec is record
    X : Integer;
  end record;

  R : Rec;

  procedure Print_R is
  begin
    Put("R.X: "); Put(R.X); New_Line;
  end Print_R;

  procedure CheckR(givenR : in out Rec) is
  begin
    Put("In CheckR"); New_Line;
    givenR.X := R.X + 1;
    Print_R;
  end CheckR;

begin
  R.X := 5;
  CheckR(R);
  Put("CheckR done"); New_Line;
  Print_R;
end Pass_By_Value;
```

Pass By Value

```
In CheckR
R.X:          5
CheckR done
R.X:          6
```

What does this line ->
print?

```
procedure Pass_By_Value is
  type Rec is record
    X : Integer;
  end record;

  R : Rec;

  procedure Print_R is
  begin
    Put("R.X: "); Put(R.X); New_Line;
  end Print_R;

  procedure CheckR(givenR : in out Rec) is
  begin
    Put("In CheckR"); New_Line;
    givenR.X := R.X + 1;
    Print_R;
  end CheckR;

begin
  R.X := 5;
  CheckR(R);
  Put("CheckR done"); New_Line;
  Print_R;
end Pass_By_Value;
```

Pass By Value

```
In CheckR  
R.X: 5  
CheckR done  
R.X: 6
```

What does this line ->
print?

```
procedure Pass_By_Value is  
  type Rec is record  
    X : Integer;  
  end record;  
  
  R : Rec;  
  
  procedure Print_R is  
  begin  
    Put("R.X: "); Put(R.X); New_Line;  
  end Print_R;  
  
  procedure CheckR(givenR : in out Rec) is  
  begin  
    Put("In CheckR"); New_Line;  
    givenR.X := R.X + 1;  
    Print_R;  
  end CheckR;  
  
begin  
  R.X := 5;  
  CheckR(R);  
  Put("CheckR done"); New_Line;  
  Print_R;  
end Pass_By_Value;
```

Pass By Reference



THE UNIVERSITY OF

MELBOURNE

Pass By Reference

```
procedure Pass_By_Reference is
  type Rec is record
    X : Integer;
  end record;

  type RecPtr is access all Rec;

  R : aliased Rec;

  procedure Print_R is
  begin
    Put("R.X: "); Put(R.X); New_Line;
  end Print_R;

  procedure CheckR(givenR : in RecPtr) is
  begin
    Put("In CheckR"); New_Line;
    givenR.X := R.X + 1;
    Print_R;
  end CheckR;

begin
  R.X := 5;
  CheckR(R'Access);
  Put("CheckR done"); New_Line;
  Print_R;

end Pass_By_Reference;
```

Pass By Reference

```
In CheckR
R.X:          6
CheckR done
R.X:          6
```

```
procedure Pass_By_Reference is
  type Rec is record
    X : Integer;
  end record;

  type RecPtr is access all Rec;

  R : aliased Rec;

  procedure Print_R is
  begin
    Put("R.X: "); Put(R.X); New_Line;
  end Print_R;

  procedure CheckR(givenR : in RecPtr) is
  begin
    Put("In CheckR"); New_Line;
    givenR.X := R.X + 1;
    Print_R;
  end CheckR;

begin
  R.X := 5;
  CheckR(R'Access);
  Put("CheckR done"); New_Line;
  Print_R;

end Pass_By_Reference;
```

Pass By Reference

```
In CheckR
R.X:          6
CheckR done
R.X:          6
```

```
procedure Pass_By_Reference is
  type Rec is record
    X : Integer;
  end record;

  type RecPtr is access all Rec;

  R : aliased Rec;

  procedure Print_R is
  begin
    Put("R.X: "); Put(R.X); New_Line;
  end Print_R;

  procedure CheckR(givenR : in RecPtr) is
  begin
    Put("In CheckR"); New_Line;
    givenR.X := R.X + 1;
    Print_R;
  end CheckR;

begin
  R.X := 5;
  CheckR(R'Access);
  Put("CheckR done"); New_Line;
  Print_R;

end Pass_By_Reference;
```

Pass By Reference

```
In CheckR
R.X:          6
CheckR done
R.X:          6
```

```
procedure Pass_By_Reference is
  type Rec is record
    X : Integer;
  end record;

  type RecPtr is access all Rec;
  -- pointer type

  R : aliased Rec;

  procedure Print_R is
  begin
    Put("R.X: "); Put(R.X); New_Line;
  end Print_R;

  procedure CheckR(givenR : in RecPtr) is
  begin
    Put("In CheckR"); New_Line;
    givenR.X := R.X + 1;
    Print_R;
  end CheckR;

begin
  R.X := 5;
  CheckR(R'Access);
  Put("CheckR done"); New_Line;
  Print_R;

end Pass_By_Reference;
```

Pass By Reference

```
In CheckR
R.X:          6
CheckR done
R.X:          6
```

```
procedure Pass_By_Reference is
  type Rec is record
    X : Integer;
  end record;

  type RecPtr is access all Rec;
  R : aliased Rec;

  procedure Print_R is
  begin
    Put("R.X: "); Put(R.X); New_Line;
  end Print_R;

  procedure CheckR(givenR : in RecPtr) is
  begin
    Put("In CheckR"); New_Line;
    givenR.X := R.X + 1;
    Print_R;
  end CheckR;

begin
  R.X := 5;
  CheckR(R'Access);
  Put("CheckR done"); New_Line;
  Print_R;

end Pass_By_Reference;
```

pointer type

Pass By Reference

```
In CheckR
R.X:          6
CheckR done
R.X:          6
```

```
procedure Pass_By_Reference is
  type Rec is record
    X : Integer;
  end record;
  type RecPtr is access all Rec;
  R : aliased Rec; allows constructing pointers to R

  procedure Print_R is
  begin
    Put("R.X: "); Put(R.X); New_Line;
  end Print_R;

  procedure CheckR(givenR : in RecPtr) is
  begin
    Put("In CheckR"); New_Line;
    givenR.X := R.X + 1;
    Print_R;
  end CheckR;

begin
  R.X := 5;
  CheckR(R'Access);
  Put("CheckR done"); New_Line;
  Print_R;

end Pass_By_Reference;
```

Pass By Reference

```
In CheckR
R.X:          6
CheckR done
R.X:          6
```

```
procedure Pass_By_Reference is
  type Rec is record
    X : Integer;
  end record;
  type RecPtr is access all Rec;
  R : aliased Rec;
  allows constructing pointers to R

  procedure Print_R is
  begin
    Put("R.X: "); Put(R.X); New_Line;
  end Print_R;

  procedure CheckR(givenR : in RecPtr) is
  begin
    Put("In CheckR"); New_Line;
    givenR.X := R.X + 1;
    Print_R;
  end CheckR;

begin
  R.X := 5;
  CheckR(R'Access);
  Put("CheckR done"); New_Line;
  Print_R;

end Pass_By_Reference;
```


Pass By Reference

```
In CheckR
R.X:      6
CheckR done
R.X:      6
```

```
procedure Pass_By_Reference is
  type Rec is record
    X : Integer;
  end record;
  type RecPtr is access all Rec;
  R : aliased Rec; allows constructing pointers to R

  procedure Print_R is
  begin
    Put("R.X: "); Put(R.X); New_Line;
  end Print_R;

  procedure CheckR(givenR : in RecPtr) is
  begin
    Put("In CheckR"); New_Line;
    givenR.X := R.X + 1;
    Print_R;
  end CheckR;

begin
  R.X := 5;
  CheckR(R'Access);
  Put("CheckR done"); New_Line;
  Print_R;

end Pass_By_Reference;
```

pointer type

takes a pointer as its argument

Pass By Reference

```
In CheckR
R.X:          6
CheckR done
R.X:          6
```

```
procedure Pass_By_Reference is
  type Rec is record
    X : Integer;
  end record;
  type RecPtr is access all Rec;
  R : aliased Rec;
  allows constructing pointers to R

  procedure Print_R is
  begin
    Put("R.X: "); Put(R.X); New_Line;
  end Print_R;
  takes a pointer as its argument

  procedure CheckR(givenR : in RecPtr) is
  begin
    Put("In CheckR"); New_Line;
    givenR.X := R.X + 1;
    Print_R;
  end CheckR;

begin
  R.X := 5;
  CheckR(R'Access);
  Put("CheckR done"); New_Line;
  Print_R;

end Pass_By_Reference;
```

Pass By Reference

```
In CheckR
R.X:          6
CheckR done
R.X:          6
```

```
procedure Pass_By_Reference is
  type Rec is record
    X : Integer;
  end record;

  type RecPtr is access all Rec;
  R : aliased Rec;

  procedure Print_R is
  begin
    Put("R.X: "); Put(R.X); New_Line;
  end Print_R;

  procedure CheckR(givenR : in RecPtr) is
  begin
    Put("In CheckR"); New_Line;
    givenR.X := R.X + 1;
    Print_R;
  end CheckR;

begin
  R.X := 5;
  CheckR(R'Access);
  Put("CheckR done"); New_Line;
  Print_R;
end Pass_By_Reference;
```

pointer type

allows constructing pointers to R

takes a pointer as its argument

constructing a pointer to R

Pass By Reference

```
In CheckR
R.X:          6
CheckR done
R.X:          6
```

Lets look at the
similar code in Java

```
procedure Pass_By_Reference is
  type Rec is record
    X : Integer;
  end record;
  type RecPtr is access all Rec;
  R : aliased Rec;
  allows constructing pointers to R

  procedure Print_R is
  begin
    Put("R.X: "); Put(R.X); New_Line;
  end Print_R;
  takes a pointer as its argument

  procedure CheckR(givenR : in RecPtr) is
  begin
    Put("In CheckR"); New_Line;
    givenR.X := R.X + 1;
    Print_R;
  end CheckR;
  constructing a pointer to R

begin
  R.X := 5;
  CheckR(R'Access);
  Put("CheckR done"); New_Line;
  Print_R;
end Pass_By_Reference;
```

Equality - Pass by Value



```
procedure Pass_By_Value is
  type Rec is record
    X : Integer;
  end record;

  R : Rec;

  procedure CheckR(givenR : in out Rec) is
  begin
    givenR.X := R.X + 1;
    if GivenR = R then
      Put("Equal");
    else
      Put("Not Equal");
    end if;
    New_Line;
  end CheckR;

begin
  R.X := 5;
  CheckR(R);
end Pass_By_Value;
```

Equality - Pass by Value



THE UNIVERSITY OF
MELBOURNE

```
procedure Pass_By_Value is
  type Rec is record
    X : Integer;
  end record;

  R : Rec;

  procedure CheckR(givenR : in out Rec) is
  begin
    givenR.X := R.X + 1;
    if GivenR = R then
      Put("Equal");
    else
      Put("Not Equal");
    end if;
    New_Line;
  end CheckR;

begin
  R.X := 5;
  CheckR(R);
end Pass_By_Value;
```

Prints "Not Equal"

Equality - Pass by Reference



THE UNIVERSITY OF
MELBOURNE

```
procedure Pass_By_Reference is
  type Rec is record
    X : Integer;
  end record;

  type RecPtr is access all Rec;

  R : aliased Rec;

  procedure CheckR(givenR : in RecPtr) is
  begin
    givenR.X := R.X + 1;
    if GivenR = R'Access then
      Put("Equal");
    else
      Put("Not Equal");
    end if;
    New_Line;
  end CheckR;

begin
  R.X := 5;
  CheckR(R'Access);
end Pass_By_Reference;
```

Equality - Pass by Reference



THE UNIVERSITY OF
MELBOURNE

```
procedure Pass_By_Reference is
  type Rec is record
    X : Integer;
  end record;

  type RecPtr is access all Rec;

  R : aliased Rec;

  procedure CheckR(givenR : in RecPtr) is
  begin
    givenR.X := R.X + 1;
    if GivenR = R'Access then
      Put("Equal");
    else
      Put("Not Equal");
    end if;
    New_Line;
  end CheckR;

begin
  R.X := 5;
  CheckR(R'Access);
end Pass_By_Reference;
```

Prints "Equal"

Reasoning about Pointers



- Why it is difficult
 - Pointers make program reasoning difficult
 - Multiple references can point to the same memory
 - This creates aliases
 - Aliases complicate reasoning

Reasoning about Pointers

- Why it is difficult
 - Pointers make program reasoning difficult
 - Multiple references can point to the same memory
 - This creates aliases
 - Aliases complicate reasoning

In the presence of aliasing explicit updates have implicit effects, hence some facts do not remain valid anymore

Reasoning about Pointers



- Though difficult, lets see how could we reason about programs with pointers

Reasoning about Pointers



- Though difficult, lets see how could we reason about programs with pointers
- Reasoning about aliases is ambiguous
 - Check to ensure that you do not have active aliases in your programs with pointers

Reasoning about Pointers

- Though difficult, lets see how could we reason about programs with pointers
- Reasoning about aliases is ambiguous
 - Check to ensure that you do not have active aliases in your programs with pointers
- We are only allowed to have aliases in our Ada programs, if we use only one of them at a time